TP 1 : SYSTÈME DE GESTION DE BIBLIOTHÈQUE

Durée : 2 heures

OBJECTIFS PÉDAGOGIQUES

À la fin de ce TP, l'étudiant sera capable de :

- Utiliser les structures de contrôle (conditions, boucles)
- Manipuler les tableaux
- Implémenter la surcharge de méthodes
- Appliquer les concepts d'héritage et de polymorphisme
- Développer une application Java complète

CONTEXTE DU PROBLÈME

Vous êtes chargé de développer un système de gestion pour une bibliothèque universitaire. Ce système doit permettre de gérer les livres, les utilisateurs (étudiants et professeurs) et les opérations d'emprunt/retour.

Fonctionnalités attendues :

- Gestion d'un catalogue de livres
- Inscription d'utilisateurs (étudiants et professeurs)
- Système d'emprunt avec règles différenciées
- Recherche de livres (par titre, auteur, genre)
- Affichage des statistiques

PARTIE 1 : CLASSE LIVRE

Objectif : Créer la classe de base pour représenter un livre.

```
public class Livre {
    // Attributs
    private String titre;
    private String auteur;
    private String isbn;
    private String genre;
    private boolean disponible;
    private int anneePublication;

    // TODO 1.1 : Créer le constructeur avec tous les paramètres
```

```
    // TODO 1.2 : Créer les getters et setters

    // TODO 1.3 : Implémenter la méthode toString()

    // TODO 1.4 : Créer une méthode emprunter() qui change le statut à non
disponible
    // Condition : le livre doit être disponible

    // TODO 1.5 : Créer une méthode retourner() qui remet le livre disponible
}
```

Questions :

1. Pourquoi utilise-t-on des attributs private ?

2. Que se passe-t-il si on essaie d'emprunter un livre déjà emprunté ?

PARTIE 2 : GESTION DU CATALOGUE

Objectif : Créer une classe pour gérer une collection de livres avec des opérations de recherche.

```
public class Catalogue {
    private Livre[] livres;
    private int nombreLivres;
    private static final int TAILLE_MAX = 100;

    // TODO 2.1 : Créer le constructeur qui initialise le tableau

    // TODO 2.2 : Méthode pour ajouter un livre
    public boolean ajouterLivre(Livre livre) {
        // Vérifier s'il y a encore de la place
        // Ajouter le livre si possible
        // Retourner true si ajouté, false sinon
    }

    // TODO 2.3 : Surcharge - Rechercher par titre
    public Livre rechercher(String titre) {
        // Parcourir le tableau avec une boucle for
        // Comparer les titres
        // Retourner le livre trouvé ou null
    }

    // TODO 2.4 : Surcharge - Rechercher par titre et auteur
    public Livre rechercher(String titre, String auteur) {
        // Logique similaire mais avec deux critères
    }

    // TODO 2.5 : Surcharge - Rechercher par année
    public Livre[] rechercher(int anneePublication) {
        // Retourner un tableau de tous les livres de cette année
        // Utiliser une boucle pour compter d'abord, puis créer le tableau
    }

    // TODO 2.6 : Méthode pour lister tous les livres disponibles
    public void afficherLivresDisponibles() {
```

```
        // Parcourir avec une boucle for-each
        // Afficher seulement les livres disponibles
    }

    // TODO 2.7 : Méthode pour compter les livres par genre
    public void statistiquesParGenre() {
        // Créer un tableau pour compter les occurrences
        // Utiliser des conditions pour classifier les genres
    }
}
```

Questions :

1. Pourquoi utiliser plusieurs versions de la méthode rechercher() ?

2. Comment optimiser la recherche dans un grand catalogue ?

PARTIE 3 : HÉRITAGE - CLASSES UTILISATEURS

Objectif : Utiliser l'héritage pour créer différents types d'utilisateurs.

```
// TODO 3.1 : Classe parent Utilisateur
public abstract class Utilisateur {
    protected String nom;
    protected String prenom;
    protected String numeroId;
    protected Livre[] livresEmpruntes;
    protected int nombreEmprunts;

    // TODO : Constructeur, getters, setters

    // Méthode abstraite - chaque type d'utilisateur aura sa propre règle
    public abstract boolean peutEmprunter();
    public abstract int getLimiteEmprunt();

    // Méthode commune
    public boolean emprunterLivre(Livre livre) {
        // TODO : Implémenter la logique d'emprunt
        // Vérifier si l'utilisateur peut emprunter
        // Vérifier si le livre est disponible
        // Ajouter le livre à la liste des emprunts
    }
}

// TODO 3.2 : Classe Etudiant hérite de Utilisateur
public class Etudiant extends Utilisateur {
    private String niveauEtude; // "Licence", "Master", "Doctorat"

    // TODO : Constructeur qui appelle super()

    @Override
    public boolean peutEmprunter() {
        // Règle : maximum 3 livres pour les étudiants
        return nombreEmprunts < 3;
    }

    @Override
```

```
    public int getLimiteEmprunt() {
        return 3;
    }

    // TODO 3.3 : Surcharger la méthode toString()
}

// TODO 3.4 : Classe Professeur hérite de Utilisateur
public class Professeur extends Utilisateur {
    private String departement;
    private boolean estTitulaire;

    // TODO : Constructeur

    @Override
    public boolean peutEmprunter() {
        // Règle : maximum 10 livres pour les professeurs titulaires, 5 pour les
autres
        int limite = estTitulaire ? 10 : 5;
        return nombreEmprunts < limite;
    }

    @Override
    public int getLimiteEmprunt() {
        return estTitulaire ? 10 : 5;
    }
}
```

Questions :

1. Pourquoi utiliser une classe abstraite pour Utilisateur ?

2. Que signifie le mot-clé protected ?

3. Comment fonctionne l'héritage en Java ?

PARTIE 4 : POLYMORPHISME - SYSTÈME DE BIBLIOTHÈQUE

Objectif : Créer le système principal utilisant le polymorphisme.

```
public class Bibliotheque {
    private Catalogue catalogue;
    private Utilisateur[] utilisateurs;
    private int nombreUtilisateurs;
    private static final int MAX_UTILISATEURS = 50;

    // TODO 4.1 : Constructeur

    // TODO 4.2 : Méthode pour inscrire un utilisateur
    public boolean inscrireUtilisateur(Utilisateur utilisateur) {
        // Polymorphisme : on accepte tout type d'utilisateur
    }

    // TODO 4.3 : Méthode pour effectuer un emprunt
    public boolean effectuerEmprunt(String numeroId, String titreLivre) {
        // 1. Trouver l'utilisateur par son ID
        Utilisateur utilisateur = null;
```

```java
        for (int i = 0; i < nombreUtilisateurs; i++) {
            if (utilisateurs[i].getNumeroId().equals(numeroId)) {
                utilisateur = utilisateurs[i];
                break;
            }
        }

        if (utilisateur == null) {
            System.out.println("Utilisateur non trouvé");
            return false;
        }

        // 2. Trouver le livre
        Livre livre = catalogue.rechercher(titreLivre);
        if (livre == null) {
            System.out.println("Livre non trouvé");
            return false;
        }

        // 3. Utiliser le polymorphisme pour l'emprunt
        return utilisateur.emprunterLivre(livre);
    }

    // TODO 4.4 : Méthode pour afficher les emprunts par type d'utilisateur
    public void afficherStatistiquesEmprunts() {
        int empruntsEtudiants = 0;
        int empruntsProfesseurs = 0;

        for (int i = 0; i < nombreUtilisateurs; i++) {
            Utilisateur user = utilisateurs[i];

            // Polymorphisme : instanceof pour déterminer le type
            if (user instanceof Etudiant) {
                empruntsEtudiants += user.getNombreEmprunts();
            } else if (user instanceof Professeur) {
                empruntsProfesseurs += user.getNombreEmprunts();
            }
        }

        System.out.println("Emprunts étudiants : " + empruntsEtudiants);
        System.out.println("Emprunts professeurs : " + empruntsProfesseurs);
    }

    // TODO 4.5 : Méthode pour afficher les informations de tous les utilisateurs
    public void afficherTousLesUtilisateurs() {
        // Utiliser le polymorphisme : toString() sera appelé selon le type réel
        for (int i = 0; i < nombreUtilisateurs; i++) {
            System.out.println(utilisateurs[i].toString());
        }
    }
}
```

PARTIE 5 : CLASSE PRINCIPALE ET TESTS

Objectif : Créer la classe main pour tester tout le système.

```
public class TestBibliotheque {
    public static void main(String[] args) {
        // TODO 5.1 : Créer une instance de Bibliothèque
        Bibliotheque biblio = new Bibliotheque();

        // TODO 5.2 : Ajouter des livres au catalogue
        // Créer au moins 5 livres de genres différents

        // TODO 5.3 : Créer et inscrire des utilisateurs
        // Au moins 2 étudiants et 2 professeurs

        // TODO 5.4 : Tester les emprunts
        // Tenter des emprunts valides et invalides

        // TODO 5.5 : Tester les recherches
        // Rechercher par différents critères

        // TODO 5.6 : Afficher les statistiques
        // Utiliser toutes les méthodes d'affichage

        // TODO 5.7 : Test des limites
        // Faire emprunter plus que la limite autorisée à un étudiant
    }

    // TODO 5.8 : Méthode utilitaire pour créer des données de test
    public static void creerDonneesTest(Bibliotheque biblio) {
        // Utiliser des boucles pour créer plusieurs objets
    }
}
```

EXTENSIONS POSSIBLES

Si vous finissez avant la fin du TP :

TODO Bonus 1 : Ajouter une classe EmpruntHistorique avec date d'emprunt et de retour

TODO Bonus 2 : Implémenter une méthode de tri des livres par année ou auteur

LIVRABLES ATTENDUS

1. Tous les fichiers .java commentés

2. Réponses aux questions posées dans chaque partie